Hydrofrac: inventory

What exists, what a result is worth, and the handful of ideas that make it work

Hydro's type system certifies **consistent computation**: a cluster collection labelled `EventualConsistency` (`Cluster<'a, Tag, EventualConsistency>`) is one whose members compute the same contents when given the same inputs, because every operator on the path is a deterministic function of its input *set* (order- and retry-insensitive). That guarantee has an unstated premise: *same inputs*. A `broadcast_closed` over `fail_stop` mints the label on the grounds that "a failed connection models the recipient as having failed"; a sender that dies between the second and third send of one fan-out leaves two members with different input sets, and every downstream operator is perfectly consistent with its own, wrong, input. **Hydrofrac discharges that premise**: it searches the program's declared fault model (crashes of registered nodes, omissions on lossy channels) for a fault set and schedule under which correct members end the run holding different facts. The label and the certificate together are convergence under faults; neither alone is. Nothing else in the toolchain has the output shape this needs: the type system cannot see which inputs arrive, the simulator explores interleavings but has no faults in its state space, and a random fault injector (Maelstrom, Jepsen) produces "ran for a while, saw nothing", which is not a completeness claim.

This document is an inventory written after five rounds of development. Part I is the artifact: how to run it and exactly what each verdict is worth. Part II is the load-bearing ideas. The authoritative description is `docs/design/hydrofrac.md`; the reasoning behind its choices is in `docs/design/history-hydrofrac-development.md` (five rounds; §9 findings, §10 limits, §12 status) and `docs/design/history-hydrofrac-negation.md`. Where the history and the design doc disagree, this document follows the design doc and says so.

> **Status note.** This inventory was synthesized from the design and history docs on branch `ldfi-hydrofrac` (tip dated 2026-10-18). Later, unmerged Hydrofrac work exists on the sibling sandbox branch `sandbox-6479bd3c-b09a-4092-b96d-1375d67833f6` (dated 2026-10-19): removal of `Level::Slice`/`Counterfactual` (`4ddea33f19`), Record→Precise with replay everywhere (`fafcd70fe3`), two clause-soundness fixes that had certified ISR's bug at 3 of 4 levels (`4c43956b4b`), measured disjunctive provenance recommended off

> (`73a5db9c05`), and `docs/design/ldfi-level-ablation.md`. Part II describes the `ldfi-hydrofrac` design; see the "Unmerged sibling work" entries in the Appendix for the differences. Note also that the `ldfi-hydrofrac` tip deletes the sedge/cress crates relative to `feat/sedge-overload-harness`, likely a divergence artefact to resolve before merging.

PART I — The artifact

What it checks

For every collection the program labels `EventualConsistency`, hydrofrac generates one specification (`Spec::eventual_consistency`, view in `hydro_lang/src/spec_views.rs`) and runs one lineage-driven search for a fault set that breaks it. The specification is:

1. **Probe.** Run the body with no faults. Collect every fact any member of the collection holds at the end into a fixed set, `FACTS`. If `FACTS` is empty, the verdict is `Vacuous`.
2. **Per faulty run.** Exclude the members the hypothesis crashes. For each fact in `FACTS`, if any surviving member holds it, every surviving member must hold it. As a count: `|distinct (member, fact) pairs at correct members over FACTS| = |correct| × |FACTS present|`.

Three correct members ending with `{1,2}`, `{1,2}`, `{1}` is the bug. `{1}`, `{1}`, `{1}` is fine: a fault may have kept item `2` out of the system altogether; EC requires convergence among survivors, not loss-free delivery. New facts that appear only in faulty runs are outside the domain and create no obligation (Part II, idea 5, says why that is essential rather than lazy).

How to run it

Automatic path (`Spec::all_labels` + `Ldfi::search_all_labels`). The program needs no hooks and no observation points; the test supplies topology, workload, crashable nodes, crash budget and a body. This is `hydro_test/src/cluster/ec_all_labels_classes.rs`, lightly trimmed:

```
use hydro_lang::sim::ldfi::{LabelVerdict, Ldfi, Level, NotCheckedReason,
Outcome};
```

```
use hydro_lang::sim::spec::Spec;

let mut flow = FlowBuilder::new();
let source = flow.process::<Source>();
let members = flow.cluster::<Member>();
let (send, items) = source.sim_input::<u32, TotalOrder, ExactlyOnce>();

// Minted EventualConsistency by the transport; nothing here mentions hydrofrac.
let at_members = items.broadcast_closed(&members,
TCP.fail_stop().name("bcast").bincode());
let _out = at_members.weaken_consistency().sim_cluster_output();

let spec = Spec::all_labels(&mut flow);          // names no collection

let compiled = flow.sim()
    .skip_consistency_assertions()               // the simulator does not check
the labels itself
    .with_cluster_size(&members, 3)
    .with_provenance()                           // needed for Level::Record
    .compiled();

let results = Ldfi::new()
    .level(Level::Record)
    .crashable("source", &source)
    .crashable_cluster("member", &members, 0..3)
    .crash_budget(1)
    .spec(&spec)
    .search_all_labels(&compiled, async |_run| {
        send.send(1);
        send.send(2);
        hydro_lang::sim::quiesce().await;
    });
eprintln!("{results}");
assert!(results.all_certified_or_not_checked());
```

`results` is a `LabelResults`: one `LabelResult { collection, verdict }` per labelled collection, `verdict: LabelVerdict = Searched(Outcome) | NotChecked(NotCheckedReason)`. Query per site with `results.at("file.rs:149")`, or `results.counterexamples()`. The default exploration is `Exploration::Exhaustive`; use `.exploration(Exploration::Fuzz { instances: n })` when the schedule space is too big, at the cost described below. `Level::Slice` is the default lineage level; `Level::Record` needs `.with_provenance()` and is what every EC harness uses. Knobs: `LDFI_LEVEL=exec|slice|record|replay` (via `.level_from_env()`), `LDFI_LINEAGE_DEBUG`, `LDFI_SOLVER_DEBUG`, `LDFI_REPLAY_DEBUG`. Everything is behind the `ldfi` feature of `hydro_lang`; the ILP solver (HiGHS via `good_lp`) builds from source and needs `cmake`, a C++ toolchain and `libclang`.

Which collections get a row: a node where the label is **minted** (a `broadcast_closed` over a granting transport, an `assert_has_consistency_of`) or where it **exits** the labelled region (`weaken_consistency`, a `batch` into an unlabelled tick, a program output). Both are reported because they disagree in exactly the interesting cases: reliable broadcast's first hop is minted and *violated*; its delivered set, derived from that label, *certifies* because of the echo the type system cannot see.

Hand-written path. Build `Spec::new(&mut flow)`, name what you observe (`spec.observe(..)`, `spec.observe_cluster(collection)`, `spec.observe_sequenced(..)`), name the correct members (`spec.correct_members(&cluster, 0..n)`), and either call `spec.eventual_consistency(observed, correct)` for the generated view on one collection, or write your own `spec.pre(view)` / `spec.post(view)` as Hydro over the observed facts. Then `Ldfi::search`. Harnesses: `hydro_test/src/cluster/{raft_ldfi,paxos_ldfi,two_pc_ldfi,ec_ldfi,ec_demos_ldfi}.rs`, `hydro_test/src/distributed/quorum_ldfi.rs`. The older goal API (`goal`, `goal_all_members`, `goal_witnessed` with `Run::witness` / `require`) still exists; no harness uses it.

The body is part of the specification. It decides which inputs arrive, which timers fire, and how long the protocol has to converge. Faults are admitted only before the body calls `run.end_of_finite_failures()`; the body must fire the protocol's recovery timers *after* that call, or a correct protocol "fails" (Part II, idea 6).

The verdicts

Verdict	What it is
Certified (`Outcome::Certificate`, `Grade::Generated`, `Exploration::Exhaustive`)	The ILP found no fault set within derivation, and every hypothesis
Violated (`Outcome::Counterexample`)	One admissible fault set and one check failed. Carries the faults, t every hypothesis tried.

Verdict	What it is
Fuzz, no violation (`Certificate` with `Exploration::Fuzz { instances }`)	Same search, but each hypothes sampled schedules.
Budget (`Outcome::Budget`)	Hypothesis budget (default 256,
Vacuous (`Outcome::Vacuous`)	The failure-free probe produced never held (hand-written).

Verdict	What it is
Not checked (`LabelVerdict::NotChecked`)	The collection is outside the gen `LocalInputsOnly`, `NotAStream`
RELATIVE (`Grade::HandWritten`, also `Grade::Goal`)	A `Certificate` under a hand-w

A test fails only on what it asserts: `LabelResults` is data, and nothing fails for a label the test does not look at.

What it has found

Real defects in production code, none injected (history §9.2, §9.1):

- **Paxos leader-change recommit loop** (`hydro_test/src/cluster/paxos.rs`). A new leader's recommit values came from a fixed P1b snapshot and were re-sent every tick; each P2b round triggered the next tick, so proposer and acceptors exchanged the same P2a/P2b forever. Surfaced as the first faulty hypothesis never quiescing. Fixed by gating the recommit on the leader's first tick.
- **Compartmentalized Paxos read quorum was not a column.** Election accepted any `rows` P1bs of the `rows × cols` grid, so a proposer could be elected on one row, which does not intersect the other row's write quorum. Agreement violation on the *correct* variant in 8 hypotheses, no fault needed, only a P1b ordering. Fixed: P1bs name their acceptor and are counted per column.

- **P1b quorum counted replies, not distinct acceptors.** A proposer that finds no leader re-sends the same ballot's P1a, acceptors answer every P1a, `collect_quorum` counts replies, so two replies from one acceptor made a quorum of two. Invisible until the body's `EFF` moved past the first ballot; then violated on correct Paxos in 48 hypotheses and correct compartmentalized Paxos in 18. Fixed by keeping one P1b per `(ballot, acceptor)`.
- **`hydro_std::quorum::collect_quorum_with_response` was not a function of its input** despite its `nondet!` justification (one batching emits `{a, b}`, another `{a, b, c}`). Found by exhaustive schedule exploration in two schedules once a test asked. Fixed, along with an idempotence-justified `fold` whose retry-only cast had no simulator implementation.
- **Maelstrom backend: untagged shared channel type.** Every cluster-to-cluster channel shared one message type, so two such channels deserialized each other's bytes. Found by the *first Maelstrom run* of the Raft comparison, not by hydrofrac; a code path no simulator test reaches.

Injected bugs, as calibration. Raft's election restriction comparing last-log index only (ignoring term; shipped in real implementations; passes all thirteen `raft.rs` tests) was found in **90 hypotheses**, four omissions and no crash, under a *generic* body shaped only by what correct Raft needs to converge: `{omit rpc[0→1]:AppendEntries(r0), omit rpc[0→2]:AppendEntries(r0), omit rpc[1→0]:AppendEntries(r1) ×2}`. Member 0 leads term 1 with `r0` unreplicated; member 1 leads term 2 and commits `r1` at the same index while 0 never hears of it; in the heal phase 0 campaigns with `(index 1, term 1)`, wins under the index-only check, and overwrites the committed `r1`. The author had not written this scenario down. **Honesty correction:** an earlier version reported "17 hypotheses" and a 25-vs-160-vs-random comparison. That body staged the bug's scenario by hand (who leads which term, when the stale node's timer fires), leaving the search two omissions to find. It is withdrawn; blind and random have not been re-measured on the generic body (a day of compute each). Under Maelstrom (12 runs, 29 min, partition and latency nemeses) the bug never occurred: its trigger is *selective* loss that keeps logs the same length while contents diverge, which a partition does not produce. Smaller calibrations: Paxos commit quorum of `f` instead of `f + 1`, three omissions, schedule-independent once `EFF` followed the first ballot; compartmentalized Paxos write quorum one acceptor short, 3 hypotheses; `collect_quorum` with a non-deduplicating caller, 5; tries-twice reliable delivery, 3 runs and two omissions (the paper's example).

Every producer of the label in the codebase (`hydrofrac.md` "Results"; one crash allowed; counts include the probe):

Program	Result
`broadcast_closed` process→cluster, `fail_stop`	**Violated**, 5 hyp. `crash(source after bcast[→1]:Item(2))`: sender dies mid-fan-out.
`broadcast_closed` process→cluster, `lossy_delayed_forever`	**Violated**, 2 hyp. `omit(bcast[→0]:Item(2))`: nothing retransmits; "indefinitely delayed" is never.
`broadcast_closed` cluster→cluster, `fail_stop`	**Violated**, 5 hyp.
`broadcast_live_from_process`, `leader_merge_broadcast` (ec_inference_demos)	**Violated**, 5 and 2 hyp. The demo's own prose caveat ("assumes the leader survives"), as a counterexample.
Echo reliable broadcast (`hydro_test`), `reliable_broadcast_closed`, `reliable_broadcast_live`	**Certified**, 10 hyp. each, exhaustive.
`uniform_reliable_broadcast_closed` (thresholds 1, 2), `g_set_gossip`, `g_set_gossip_live`	**Fuzz, no violation**, 28 / 13 hyp., 8 schedules per hyp.
`multi_paxos`, learners' `learned`, proposer held correct	**Fuzz, no violation**, 24–25 hyp., 8 schedules. The learner echo cycle's result, not consensus's.
`multi_paxos`, all ten labels under `Spec::all_labels`, proposer crashable	**5 violated** (single-hop mints `:281`, `:549`, `:582`, the exit `:286`, the keyed echo `:591`), **4 fuzz-no-violation** (`:569`, `:570` trivially, one proposer; `learned` `:588` and its `flat_map` `:608`), **1 not checked** (`epoch_splice.rs:190`, a `Singleton`).
ISR, client and ZK crashable, `EFF` after the first round	`:343` write broadcast **Violated** by the client's mid-fan-out crash; `:564` view broadcast and `:360` replication **Violated**, 8 hyp.: ZK dies between two sends of one view.

Program	Result
2PC with cooperative termination, exhaustive	Coordinator crashable: `:86`, `:92` **Violated**, 2 hyp. each. Coordinator held correct: **Certified**, 3 and 5 hyp. Peer channels `:98`, `:161` **Vacuous**: the failure-free run never exercises termination.
ABD (3 replicas, 1 client)	Every replica-labelled stream **Violated**, 2 hyp., by the client's crash between two sends. The register fold `:165` **Not checked**.

The line the search drew without being told where it was: *every single-hop or single-point dissemination fails under one crash; every echo or gossip cycle holds.* Both transports that `networking/mod.rs` grants `EventualConsistency` justify it by reasoning about the recipient or about delay; neither survives one sender crash or one lost message. The label is a promise about the program using the transport, and only a program with a second dissemination path keeps it.

Hand-written harnesses, graded RELATIVE: correct Raft certified in 235 hypotheses (durability of a committed entry); correct Paxos, Paxos+KV store, compartmentalized Paxos in 372, 301, 568; 2PC with cooperative termination certified under one crash, broken under two; `collect_quorum` with a deduplicating caller in 28; ISR's unclean election reproduces the Kafka bug.

Note on counts: the history (§1.2) reports "4 hypotheses" where the design doc reports "5" for the same `broadcast_closed` violations, and "Certificate, 26" for multi-Paxos where the design doc says "No violation found, 24–25, 8 sampled schedules". The design doc is current (counts include the probe; Fuzz is not a certificate) and is used above.

Explicit non-coverage

- **Simulator only.** The fault filter lives in the simulator's compilation of `Network` nodes (`sim/builder.rs`), not in the IR; nothing exists under `deploy`. Embedded (raw-element) channels carry no delivery id and are not fault points.
- **Fault vocabulary.** Omission on lossy channels and permanent crash of registered nodes, both before `EFF`. No delay, reordering, duplication, partition-with-recovery or Byzantine behaviour. A `fail_stop` connection

cannot fail on its own. A partition can be approximated by an omission set (Raft's counterexample is four omissions on two links).

- **Out of class, reported not checked:** scalar labels (`Singleton`, `Optional`, `KeyedSingleton`: ABD's register, `epoch_splice`'s fold); labels inside a tick or atomic region; collections with local inputs only; element types that are not `Clone + Serialize` (exclude with `Spec::all_labels_except`).
- **One test, one topology, one horizon per certificate.** Cluster sizes, inputs, timers and the recovery window are the body's. **Fuzz is not a certificate**; uniform broadcast, gossip, multi-Paxos, Raft and Paxos all sample schedules today. **Finite horizon**: never-converges and not-yet-converged look the same; liveness is not checked.
- **Static membership.** `_live` variants are tested for faults, not join timing; no membership-change protocol exists on the branch.
- **Precise lineage has gaps.** The provenance rewrite covers ~26 `HydroNode` variants and panics by name on others (`entries_partially_ordered` on keyed streams; `leader_merge` runs at `Level::Slice`). Replay needs hooked batches. Deduplicating operators keep one derivation per record, so alternatives surface one hypothesis at a time.
- **Tested scale.** A few hundred fault points, eight records of workload, up to eight nodes.

Cost

Search	Reported cost
Plain / reliable broadcast, 2PC (small, exhaustive)	Seconds. Observer replays ~8 per schedule, each a few ms.
EC suite (`ec_ldfi`, `ec_demos_ldfi`)	~1.5 min. Full `hydro_test` LDFI set ~7 min.
Paxos injected bug	8–50 s, 12–43 hypotheses (before `EFF` was moved); schedule-independent after.
Raft injected bug (generic body)	**~30 min**: 90 hypotheses × 4 sampled schedules = 360 instances.
Correct Raft, relative certificate	**~1 h**: 235 hypotheses, 940 instances, 365,000 counterfactual replays (624 panicked and were kept conservatively). Late hypotheses cost 3–4 min each, almost all replay.
Correct Paxos ×3 (core, +KV, compartmentalized)	**~45 min** together, 4 sampled schedules per hypothesis.

Search	Reported cost
`Level::Counterfactual` vs recorded levels	~5× wall time; 300–435 replays per search on ISR.
Uniform reliable broadcast, exhaustive	240,000 interleavings passed once; the test uses `Fuzz { instances: 8 }`.

The Raft and Paxos tests are `#[ignore]`d for runtime. Raft's `committed` under the generated specification has not been run (an hour).

PART II — What is going on: the load-bearing ideas

1. The fault point is a sender-side filter on every `Network` send, and the transport type is the fault model

Every send half the simulator emits (`sim/builder.rs`, `create_network`) calls `LineageHandles::record_sent` before anything goes on the wire; that call consults a `FaultState` and returns `SendFate::{Delivered, Omitted, Crashed}`. A delivered send carries a fresh delivery id; an omitted or crashed one produces no `Recv`. The filter is there whether or not the test asked.

The identity of a send is `SendKey { channel, from, to_member, bytes }`. An omission names one; it drops every byte-identical copy on that channel while finite failures last, so a retransmission of the same payload does not acquire a new identity (Molly's omission), while a retransmission with a different payload (Raft's heartbeat with a higher `leader_commit`) is a different message. The transport the program declared decides what the filter admits: `lossy_*` admits omission; `fail_stop` admits only the sender's crash. The program's networking declarations *are* its fault model, and the harness registers no channels.

A crash is a pattern of applications of the same filter: `crash(node after message)` means the node performs every send up to and including the named one, then every later send on every channel is `Crashed` and every external output is suppressed; it never restarts. `CrashPoint::BeforeAnySend` is the degenerate case. Because the cut is at the sender, one crash point splits a single broadcast's fan-out, which is the counterexample for every single-hop broadcast in the table.

Why not the obvious alternatives. The first version put the fault point on the receiving `batch` hook, because interposition there was free (the scripted-hook layer existed). That conflated "a place to interpose" with "a model of the network": a channel consumed without a hooked batch could not be faulted; a `tee`d channel omitted at one consumer was delivered at others; crashes needed a separate mechanism because a receiver cannot stop a sender; and the soundness of every certificate rested on the author having registered every channel. The sender-side filter discharges that rule by construction (history §11). And why name a crash point by a message rather than a send count: under a different hypothesis the node's send sequence shifts, so `send #7` names a different message, the ILP's constraint means something else in the bug's world, and the first version certified a program with a known bug this way. With a message as identity, the hypothesis means the same thing in every schedule, and if the message is never sent the node simply survives.

2. Two nested searches

Hydrofrac searches **fault hypotheses**; under each hypothesis the simulator searches **schedules** (which runnable tick executes next, what an unhooked batch releases, which version a snapshot reveals). A counterexample is a (fault set, schedule) pair. The outer loop is Molly's (`Ldfi::run`, `sim/ldfi.rs`):

1. Run the probe; establish `FACTS`; record the first successful derivation.
2. Each derivation becomes a clause: a breaking fault set must interrupt at least one delivery in it. A clause also records the crashes that were *premises* of the run it came from (a derivation seen in a world where a node was dead need not hold where it is alive), and the run's send sequences, which give crash points their meaning.
3. Build the candidate universe: for every dependency, its omission (if the channel is lossy) and the crash of its sender right before that send (if the sender is crashable). Order weakest first: omissions before crashes; within a node, later crash points before earlier ones.
4. Solve an ILP (HiGHS via `good_lp`): one binary per candidate; minimise weighted count; one cover constraint per clause; crash budget and one crash per node; a no-good per tried set; no superset of a hypothesis whose every schedule was vacuous; when feasible, leave one `PRE` derivation intact.
5. Run the hypothesis under `Exploration`. A violating schedule ⇒ `Counterexample`. Otherwise add the new derivations and repeat. Infeasible ILP ⇒ `Certificate`; budget ⇒ `Budget`.

Why nested, and why ILP. Molly fixed a synchronous schedule and did not explore reordering; Hydro protocols' outcomes can depend on tick order (Raft: whether a `RequestVote` lands before or after the old leader's `AppendEntries`), so a certificate on one schedule per hypothesis could land on the harmless interleaving and certify a known bug. Hence `Exploration::Exhaustive` is required for a certificate and `Fuzz` is reported as such. The ILP rather than SAT because "weakest first" is an objective and the budget and premise constraints are linear; same problem, more convenient solver. `Strategy::Blind` is the same loop with the cover constraints removed, the ablation that measures what lineage buys (2–7× fewer hypotheses on certificates; random injection misses 2PC bugs in 2–4 of 10 repeats at five or more participants and never terminates with a claim).

3. Four lineage levels, and every fallback points toward over-approximation

Lineage is reconstructed from the simulator's event log (`sim/runtime/lineage.rs`: `Exec`, `Released`, `Sent`, `Recv`, `Output`), not from the language, because Hydro programs are Rust closures over dataflow, not Datalog rules. The derivation of an output is the backward closure from the execution that produced it to the deliveries that supported it.

`Level`	**What it records**	**Cost / what it buys**
`Execution`	A tick execution depends on everything released into it, every arrival at its node before it, and every earlier execution of every tick at the node (persisted state, over-approximated).	Free; needs nothing from the program. Coarse: a step-closure protocol is one IR node, so everything depends on everything.
`Slice` (default)	Same, narrowed by a static slice of the IR per sink (`sim/slice.rs`): which hooks, channels and local ticks a send or output can reach, split into same-execution and carried-through-`defer_tick`.	Free. Same weakness for closure-encoded protocols.

`Level`	**What it records**	**Cost / what it buys**
`Record`	Record-level provenance by an IR rewrite (`sim/provenance.rs`, `SimFlow::with_provenance`): every element becomes `Tagged<T>` carrying the delivery ids it was computed from; `join` unions, `fold` accumulates, a send strips the tags into the `Sent` event. `Reference`-capturing closures emit `Prov::OPAQUE`.	Compile-time rewrite; keeps hypothesis counts flat as workload grows (2PC load 8: 11 hypotheses vs 73 under `Slice`). Panics by variant name on unsupported operators.
`Counterfactual`	Node-local replay (`CompiledSim::replay`): re-run one node alone with its hooked batches fed their recorded releases, delete deliveries one at a time latest-first, keep a deletion if the output survives. Result: a 1-minimal sufficient set (ddmin sense).	~5× wall time; a replay is a fresh instance of the dylib, ms each but thousands per hypothesis on Raft. Needs hooked batches. Sees through closures nothing else can; on closure-encoded ISR turns 14/27 into 6/14.

Why the fallback direction is the whole soundness story. An extra dependency in a derivation gives the solver one more fault to try (cost). A missing dependency removes a constraint the breaking hypothesis would otherwise have to satisfy, so the search can certify a program with a bug (unsoundness). Every one of the four false certificates during development was an under-approximation: a byte-only match that attributed 2PC's `commit` to its `prepare`; a replay candidate set that excluded deliveries *after* the output's execution (a later echo can substitute); a replayed tick that legitimately had nothing to do; a node that would not reproduce its output even with everything kept. So: `Counterfactual` falls back to `Record`, `Record` to `Slice`, and a replay that panics (Raft's leader indexed past its log when fed recorded inputs against diverged state, 624 times in 365,000 replays) counts as "target not produced", keeping the delivery. The panic is caught on the generated side of the dylib boundary (`sim::runtime::erase_guarded`), because unwinding across it aborts the process, which it did.

4. The specification is a Hydro flow at an observer process

A `Spec` (`hydro_lang/src/sim/spec.rs`) adds an **observer** process to the program. Observed collections are sent to it over an ordinary `fail_stop` channel and presented in the observer's tick as the accumulated set of facts so far. `PRE` and `POST` are bounded `Optional`s in that tick: a view holds if non-empty at the end of the run. The observer's ticks run eagerly and outside the schedule choice (it only consumes, so its interleaving is unobservable to the program; exploring it multiplied schedules by 64× before this rule). The fault model's own facts are fed before each run: `correct_members` (the complement of Molly's `crash` relation) and, for the generated view, the fact domain `FACTS`.

Under `Spec::all_labels`, the observer's input is a placeholder *slot*. When the flow is compiled for the simulator, an IR pass (`sim/observe.rs`) enumerates labelled collections, decides class and `NotCheckedReason`, and taps each in-class one: the node becomes a `Tee` (as `.clone()` would), the new branch is mapped to `(collection index, bincode of the element)` and sent to the observer with exactly the IR a `.send()` would build. The slot becomes the union of the taps. The pass runs *before* network compilation and *before* the provenance rewrite, so the taps are traced like any other operator and lineage flows through them. `Ldfi::search_all_labels` then runs the ordinary loop once per in-class collection, telling the observer which index is current and which members of that collection's cluster are correct, and clearing the fact domain between searches.

Why a flow and not a Rust predicate. LDFI needs the specification to have *lineage*: the derivation it negates is the derivation of `POST`, not of some concrete message the author named. The first version's goals were a Rust verdict alongside hand-named witness facts, and the two could disagree (a fact that was "ever" when it should have been "final"; a channel left unregistered); that was the main source of false certificates. Making the view a Hydro dataflow means record provenance and replay apply to it unchanged. Why fix `FACTS` from a probe: an open-ended "every member holds whatever any member holds" acquires a new obligation whenever a faulty run produces a new fact, which no earlier derivation explains, and a fresh fact is not the label's premise about the *given* inputs anyway. (The history §1.2 describes a generated `PRE` "some correct member holds some fact of `FACTS`"; the code, `spec_views::eventual_consistency`, has no `PRE`: non-vacuity is the probe having a non-empty domain. The design doc and code agree; the history is stale.)

5. Monotonicity decides what a certificate is worth

The hitting-set inference is: *a fault set that leaves every delivery of a derivation intact cannot break* `POST`. That is true only if `POST` is **monotone** in the observed facts: if a set `S` of deliveries suffices for `POST`, any superset does too. A fault set that leaves `S` intact produces some superset of `S`, and the inference needs that superset to still satisfy the view.

The generated EC view is monotone by its form: `FACTS` and `correct` are fixed by the search before the run, so the view is a universal over fixed domains whose derivation is the record provenance of every `(member, fact)` pair; keep them all and the view holds. No analysis is involved, which is why it carries `Grade::Generated` and why the certificate is "within the fault model and the explored schedules". Hand-written views can have any shape, and the everyday ones are not monotone: *agreement* is the absence of a conflict (falsified by adding a fact); *durability of the final leader's log* reads a `max()` over views (a later fact changes it); *integrity* ("nobody delivered twice") holds with an empty derivation and certifies after one run having checked nothing. The search finds bugs in these when the breaking hypothesis also happens to displace a recorded fact; a conflict that appeared without displacing one would never be proposed. Hence `Grade::HandWritten`, RELATIVE, no soundness claim.

The false ISR certificate is the concrete lesson (`history-hydrofrac-negation.md` §4). When observer-level counterfactual derivation landed, ISR's durability view, with the final leader read off `observe_sequenced(views).max()`, deleted views latest-first and never changed value (every view named replica 0), so the 1-minimal sufficient set was `{the appended write, the initial view}` with no heartbeat in it. The breaking hypothesis (lose a replication, crash the leader after the ack, let ZK promote the stale follower) kills neither; it *adds* a later view. Six hypotheses, certificate, bug present. Record provenance folds every view into the aggregate and so keeps the heartbeats that kept replica 0 the leader, and with them the crash point after the acknowledgement. So for hand-written views `spec_view` takes the **union** of the counterfactual and recorded derivations at the observer: sound where either is, minimal where both agree. (The generated view takes the recorded derivation alone, with no observer replay; its form makes that sufficient.) Why counterfactual at all: a view derived through negation (`filter_not_in`, `count() == 0`) has *no* record provenance for its emptiness, and under record provenance alone the negated agreement view certified plain broadcast after one run; replaying the observer with facts removed asks the same question of both formulations and gets identical derivations. The fourth round's syntactic polarity walk (`polarity.rs`), which tried to grade arbitrary views

monotone/non-monotone/unknown, was removed in the fifth: once the one specification hydrofrac is *for* is generated and sound by form, there is nothing left to classify, and the walk could only approximate the property anyway.

6. Where the end of finite failures falls decides what can be tested

`Run::end_of_finite_failures()` flips a switch on every filter and records the event-log index. From then on omissions are ignored, and only sends recorded *before* that index are omission candidates in later hypotheses. Crash points lie before it too: `crash(node after m)` is proposed only for pre-`EFF` sends `m`. Every post-`EFF` dependency is killed by exactly the crashes that kill its sender's last pre-`EFF` send, which is how the solver sees it.

Two consequences, both learned the hard way. First, **what is a fault point is the body's decision.** Paxos's P1b-counting defect was invisible for as long as the body's `EFF` preceded the new leader's first ballot: the first round of P1bs always arrived, and the quorum was distinct acceptors' by luck of ordering. Moving `EFF` past the first check made acceptor 0's P1b a fault point, made the injected bug's counterexample schedule-independent, and exposed the real defect. The ISR table shows the same thing in miniature: with `EFF` right after the write, no ZK send is a crash point and the view broadcast "passes"; with `EFF` after the first round, it is violated. A harness that cannot lose a message cannot test the protocol's handling of its loss. Second, **the window after `EFF` must be a real recovery window.** The first version allowed crashes at any send and produced a "violation" of correct Raft in which the leader crashed in the heal phase after the last election timer had fired, and the leaderless cluster was blamed. Raft's generic body went through four versions that "violated" correct Raft before the fifth gave it enough post-`EFF` election timers, same-term requests and heartbeat rounds to converge; none of those changes was in response to the buggy protocol. The history's §10 calls `EFF` "a leftover that should not shape the design further"; the design doc lists it among the declared fault bounds a certificate is relative to. The latter is the current position, and §9.2 of the same history is the reason. `[NEEDS AUTHOR REASONING: the long-term intent for EFF: a body-supplied bound forever, or something the convergence witness (open door 2) would replace?]`

7. The simulator is a counterfactual oracle, and that is why hooks still matter

A simulated run is a function of a finite sequence of nondeterministic decisions, the scheduler's (which tick, what a batch releases) and the environment's (each send's fate); everything between is deterministic. Lineage, properly stated, is "which decisions does this fact depend on", and the simulator answers by re-running with one decision changed. Node-level replay (idea 3, and the observer replay of idea 5) is the coarse instance: the only decision it can alter is "which recorded releases this node's hooked batches made", which is why it needs hooks and why `RaftHooks` survive the move to the sender-side filter. The same determinism is what makes the outer loop valid: "the outcome still held, so the program has another derivation" is only an inference if the run differed from the last by the fault set alone. The decision-trace open door generalises this.

Open doors

1. **Scalar labels checked at their input stream.** A labelled `Singleton` / `Optional` / `KeyedSingleton` is a function of a stream the type system already vouches for; the only premise left is that every correct member receives the same inputs. Tap the labelling operator's input and report under the scalar's site. ABD's register (`abd.rs:165`) is the first case; expected to be violated by a client crash between two sends. "Final values equal" is explicitly *not* planned: it re-proves the type system's conclusion. (The history §12 proposed exactly that lattice-equality variant; the design doc's later commit `7d58ad20e0` supersedes it.)
2. **Convergence witnesses in `PRE`.** Let a protocol that can report "settled" (a leader whose view every correct member shares and whose latest heartbeat all have acknowledged) make unconverged runs vacuous rather than counterexamples, instead of tuning body length. Raft would need to expose the heartbeat acknowledgement as an output.
3. **Decision-trace replay** as the general counterfactual oracle: record every draw with its outcome, replay a prefix, override one decision, continue under a policy for the rest. Gives why and why-not at any granularity, removes Level 3's dependence on hooks, and yields a decision sequence rather than entropy bytes as a counterexample's schedule. The continuation policy after an override is the one new piece of design. Does not repair the inference for non-monotone views.

4. **Raft's `committed` under the generated specification.** The in-class form of the durability harness; carries the sound grade; the harness exists; about an hour; has not been run.
5. **Membership change.** Historically the richest vein (joint consensus, ZooKeeper reconfiguration anomalies); nothing on the branch changes configuration while running. Needs membership events as a driven `sim_input`, a decision whether a crashed member is also `Left`, and a specification of agreement across an epoch boundary.
6. **Smaller items from the roadmap**: replay cost on long runs (do not minimise over post-`EFF` deliveries; cache replays across hypotheses sharing a script prefix; neither changes a verdict); the filter as an IR node so the fault model exists under `deploy`, with delay, reordering and partition at the same filter, and a receive-side source so replay needs no hooks; unmeasured baselines (blind and random to the certificate on correct Raft, a coverage-guided fuzzer, Raft under load); making "every label has a certificate" a failing check, which needs a registry of specifications per compiled program.

Appendix: Closed doors

Hydrofrac went through five rounds, and most of what the first three built is gone. Some doors below were ones LDFI's authors had already looked behind (clock-indexed crash points, De Morgan why-not provenance); others exist only because Hydro is a Rust dataflow with a simulator rather than a Datalog with a fixed schedule (the hook as fault point, the step closure as a lineage wall, the dylib boundary). Nearly all are dead ends; nearly every one fixed a limit worth knowing. Sources: `history-hydrofrac-development.md` (H §n), `history-hydrofrac-negation.md` (N §n), `ldfi-on-sim-hooks.md` (S, by section title), and commits on `ldfi-hydrofrac`.

The three walls

Read together, the entries below are not twenty-odd separate mistakes. Nearly every door closed against one of three walls, each a limit of the setting rather than of the attempt.

- **Wall 1 — the wrong fault point.** A fault injected anywhere but the sender, or named by anything but the message itself, is not the fault Molly reasons about: its identity shifts between hypotheses, or it misses channels, or it explores a dimension the program cannot see. *Consumer-side hooks; crash points named by send count; message identity by occurrence counting;*

every observed send as a crash candidate; exploring the observer's schedule; the earlier proposal (`lfdi-on-sim-hooks.md`); Maelstrom and partition nemeses.

- **Wall 2 — unsound inference on a non-monotone view.** The hitting-set argument says "derivation intact ⇒ view holds"; that is true only for views monotone in their intervenable facts, and only when the derivation itself is attributed soundly. Every false certificate came from here. *Round-1 lineage shortcuts; the static slice; record provenance alone; counterfactual lineage alone; the syntactic polarity check; agreement as the Raft property; letting a replay panic; replay shortcuts; the lattice-aware specification; the unmerged sibling work.*

- **Wall 3 — the harness doing the protocol's job.** When the body arranges what the protocol should earn — convergence, a leader, a recovery window, a second statement of the property — the verdict is about the author's knowledge, not the protocol. *Crashes admissible at any send; hand-named goals; the staged Raft body; tuning the body; silent certification when `PRE` never held.*

Two entries fit none: *measurement mistakes* and *the undischarged-premise lint* are about how the claim was measured and scoped, not about the mechanism. "Doors we never opened" are not closed doors.

Consumer-side hooks as the fault point

- **Tried:** Round 1 (squashed into `cd6cae7fc6`; S "Prototype status"). Standing `OmissionPolicy` / `PassthroughPolicy` on the receiving `BatchHook`, channels registered by the harness. Hypothesis: the scripted-hook layer already interposes on every tick input and crosses the dylib boundary, so faults there are free. The first cut, scripting `release_values` + `pause()` from the harness, died earlier still: a schedule, not a policy, so the harness had to predict arrival order; and hold is not drop (S "Why that is the wrong fault point").
- **Closed because:** a test seam, not a model of the network (H §11). A `tee`d channel omitted at one consumer was delivered at the others; hook-free paths (`recv.map(f).send(..)`, `sim_output`) could not be faulted; crashes needed a separate mechanism in the send half; every faultable batch was a hook parameter in the protocol's signature; and every certificate rested on the author having registered every channel. Replaced by the sender-side filter in `ab3ad7fe82`.

- **What it bought us:** the filter on every `Network` send with the transport type as the fault model and every channel a fault point by construction (H §5); the distinction between interposition and schedule control, of which LDFI needs only the second. Hooks survive as optional lineage precision and as what Level 3 replay feeds.
- **Reopen only if:** never — the sender is where Molly defines omission and crash.

Crash points named by send count

- **Tried:** Round 1: `crash(node after k sends)` (S "Two soundness problems", 1; H §5.3).
- **Closed because:** under another hypothesis the node's send sequence shifts, `send #7` names a different message, the clause's literals mean something else in the bug's world, and 2PC at load 2 was certified with the bug present.
- **What it bought us:** `crash(node after sending m)`, `m` identified by `(channel, sender, destination, bytes)`; a node whose `m` is never sent survives; clauses carry the run's send sequences so the kill relation is evaluated in the run that produced them (H §7).
- **Reopen only if:** never — Molly's logical clock is the stable identity, and the message is the closest thing the simulator has to one.

Message identity by occurrence counting

- **Tried:** S "Message identity and time": `(channel, from, to, payload, k)`, retransmissions distinct faults.
- **Closed because:** ordinals are unstable across hypotheses for the same reason send counts are. The built identity is the bytes; a byte-identical retransmission is omitted too while the fault stands (H §5.2), and a different payload (Raft's heartbeat with a higher `leader_commit`) is a different fault.
- **What it bought us:** stable omission identity with no clock. [NEEDS AUTHOR REASONING: was "an identical retransmission is the same fault" accepted deliberately? A protocol retrying with identical bytes can only be tested for "all copies lost", never "first lost, retry delivered".]
- **Reopen only if:** a target retransmits byte-identical payloads and the bug needs one copy lost and another delivered.

Crashes admissible at any send

- **Tried:** Round 1 proposed crash points for every send of the run; `EFF` was implemented by swapping every hook's policy to `deliver_all()` (H §5.4, §11.1).
- **Closed because:** the swap became a switch on the filter; post-`EFF` crashes "violated" durability on *correct* Raft — the leader died in the heal phase after the last timer had fired, and a leaderless cluster was blamed for an entry it had no timer left to replicate. A crash inside the convergence window needs a second window the body cannot supply without knowing where the crash fell.
- **What it bought us:** one convergence window; crash points proposed only for pre-`EFF` sends, and every post-`EFF` dependency killed by the crashes that kill its sender's last pre-`EFF` send. (Sibling work, below, found the simulator still *fired* post-`EFF` points and closed that too.)
- **Reopen only if:** the body can express a recovery phase triggered by the crash itself.

Round-1 lineage shortcuts: "everything delivered", byte-only matching

- **Tried:** Level 0 as "the whole delivered set is the derivation" (S "Level 0"); hook releases matched to sends by bytes alone.
- **Closed because:** Level 0 put reliable delivery's acknowledgements into every support although nothing read them; byte matching attributed 2PC's `commit` to the `prepare` with the same payload, a false certificate (H §6.1).
- **What it bought us:** `Level::Execution`, still the fallback at every level; channel-scoped matching for hooked releases; delivery ids minted at the send and carried on the wire, so receive-side attribution is exact (H §11.4).
- **Reopen only if:** never.

The static slice as the precision instrument

- **Tried:** Round 1, `sim/slice.rs`: a per-sink backward slice of the IR to narrow "every output depends on every input of every earlier execution". Two completeness bugs each manufactured a false certificate (a `forward_ref`-fed hook with no channel; ISR's `Reference`-captured state invisible to the walk).
- **Closed because:** a step closure over a state struct (`raft_step`, `replica_step`) is one IR node, so the slice is Level 0 again; and even on dataflow-style ISR, Levels 0 and 1 coincide at every load — the slice cannot

separate records that share a hook (S "Ablation"). A correctness requirement for Level 1, not a search improvement.
- **What it bought us:** the diagnosis that ISR's hypothesis count was half coarse lineage and half the goal's dependence on liveness signals (every heartbeat kept the leader), which no precision removes; and that record provenance, not slicing, keeps counts flat under load (2PC load 8: 11 vs 73).
- **Reopen only if:** never as a level (removed outright on the sibling branch).

Every observed send as a crash candidate

- **Tried:** S "Two soundness problems", 2. Level 2 certified ISR at load 2 because the leader's crash *between* two acknowledgements was not in the goal's lineage — the goal was conditioned on the acknowledgement, not derived from it. First fix: admit every observed send.
- **Closed because:** it made the number of singleton hypotheses the number of sends, independent of lineage (2PC load 8: 46 instead of 11).
- **What it bought us:** Molly's answer — the precondition's lineage widens the universe: the crash right after each send in a `PRE` derivation is a *preserve point*, and universe = clause candidates ∪ preserve points (H §7 step 5).
- **Reopen only if:** never.

Hand-named goals: `goal`, `witness`, `witness_final`, `require`

- **Tried:** Rounds 1–2: the property stated twice, a Rust verdict in the body and named output facts for lineage (`f0dad9a851`). `witness_final` was added when ISR's final view, derived as an "ever" fact, certified the bug at load 2 (S "Two kinds of witness").
- **Closed because:** the two encodings were never cross-checked and disagreed: a forgotten `require`, an unregistered channel, or an "ever" that should have been "final" was a wrong certificate with no warning (H §8.4, §10).
- **What it bought us:** `Spec`: `PRE`/`POST` as Hydro views on an observer, so the verdict and its lineage are one object and the body only drives (H §6.5). The old API is compiled and unused. [NEEDS AUTHOR REASONING: kept as an escape hatch on purpose, or should it be deleted so no new harness is written on it?]
- **Reopen only if:** never for harnesses.

Exploring the observer's schedule

- **Tried:** The observer as an ordinary location whose ticks the explorer interleaved.
- **Closed because:** 64× more schedules on broadcast for an interleaving the program cannot observe (H §6.5).
- **What it bought us:** eager observer ticks outside the schedule choice.
- **Reopen only if:** never.

Record provenance alone for the observer's views

- **Tried:** Round 3: derive `POST` from the tags its value carried.
- **Closed because:** a view that holds by *absence* (`filter_not_in`, an empty `count`) has no contributing records; the negated agreement `POST` had derivation `{}` and certified plain broadcast after one run (N §1; `e3928ed9fb`). Molly's De Morgan rewrite was considered and rejected: the simulator is a counterfactual oracle and need not know how the view was computed (N §2–3).
- **What it bought us:** `Lineage::spec_view`, the observer replayed with facts removed one at a time; positive and negated agreement get identical derivations.
- **Reopen only if:** never for hand-written views.

Counterfactual lineage alone at the observer

- **Tried:** Immediately after: `spec_view` took only the counterfactual derivation.
- **Closed because:** ISR's durability `POST` reads the final leader off `observe_sequenced(views).max()`. Deleting views latest-first never changes the value, so the sufficient set was the write plus the *initial* view, with no heartbeat in its lineage; the breaking hypothesis (lose a replication, crash the leader after the acknowledgement, ZK promotes the stale follower) kills neither — it *adds* a view. Six hypotheses, certificate, bug present (N §4; `2dfc50feff`).
- **What it bought us:** the union of counterfactual and recorded derivations, sound where either is; and the general statement that the hitting-set inference holds only for views monotone in their intervenable facts, final-value views being the everyday counterexample (H §10).
- **Reopen only if:** the view is monotone by construction — which the generated specification is.

The syntactic polarity check

- **Tried:** Round 4, `polarity.rs` (`c55d449a20`; N §5 (b)): a static walk grading each view `Monotone` / `NonMonotone` / `Unknown` to label the certificate and license the counterfactual-only derivation.
- **Closed because:** removed in `09618ccd96`. It could only approximate (the positive agreements of broadcast, 2PC and Raft graded `Unknown`; `first()` is an unclassifiable `scan`), and it existed only because hand-written views can have any shape. The generated specification — a universal over a fact domain and a member domain the hypothesis fixes — makes "derivation intact ⇒ view holds" true by its form, leaving nothing to classify (H §1.1, §6.5).
- **What it bought us:** the grades survive as words: *within the fault model* for the generated specification, *RELATIVE, no soundness claim* for every hand-written one; N §4 is the written reason. The control case (integrity, true, non-monotone, certified after one run with nothing checked) is the example for anyone who asks why.
- **Reopen only if:** hand-written specifications become a product rather than an escape hatch; even then a decision-trace oracle does not repair the inference for non-monotone views (N §5 (d)).

Agreement as the Raft property

- **Tried:** Rounds 1–3: "no index holds two committed entries" (S "A Raft bug"; `881605b6c0`).
- **Closed because:** agreement is the *absence* of a conflict; a hypothesis breaks it by making a member commit something else, removing no recorded fact. The staged bug was found only because the same omission also displaced member 1's recorded commit (H §6.5).
- **What it bought us:** durability (`PRE`: some member committed `(i, r)`; `POST`: every correct member did), and the division of labour — agreement belongs to the simulator's own tests and Maelstrom (H §1.1).
- **Reopen only if:** never for LDFI; outside the class by definition.

The staged Raft body and the "17 hypotheses" result

- **Tried:** Rounds 1–4 reported the election-restriction bug in 25, 20, then 17 hypotheses with a body that elected member 1, raced member 0's timer against a request, ordered requests and heartbeats, and fired member 1's timer twice after `EFF` (H §9).

- **Closed because:** everything the bug needs except the two lost messages was scripted by someone who knew it; the Maelstrom comparison credited lineage for conditions the body arranged; the author reported the number without saying so. Withdrawn with the 25-vs-160-vs-random-0/5 comparison.
- **What it bought us:** a generic body shaped only by what *correct* Raft needs, never touched in response to the buggy protocol; the bug in 90 hypotheses with a four-omission, no-crash, schedule-independent counterexample the author had not written down; correct Raft certified in 235. Blind and random on this body are unmeasured (a day of compute each).
- **Reopen only if:** never — the body is part of the claim.

Tuning the body until correct Raft converged

- **Tried:** Round 5: four bodies "violated" durability on correct Raft (no post-`EFF` timer; no request of the new term; a timer spent on a re-armed suppression; too few heartbeats for the linear back-off), each lengthened; the fourth fix also changed the protocol (§5.3's conflict hint, because the long heal phase made a hypothesis an hour).
- **Closed because:** a specification error treated as a scheduling problem. `PRE` = "some member committed" is true at the first commit, so a run too short to catch up read as a bug (H §1.1).
- **What it bought us:** `PRE` must be a convergence witness, so an unconverged run is vacuous; `Outcome::Vacuous` when no hypothesis converges. The Raft harness is not yet on this pattern (needs the latest-heartbeat acknowledgement as an output; H §12 item 0).
- **Reopen only if:** never — fix `PRE`, not the horizon.

Silent certification when `PRE` never held

- **Tried:** Rounds 1–4: a failure-free run with `PRE` false contributed `{}`, the ILP was infeasible, outcome `Certificate` (H §7, §9). The ISR false certificate of round 1 had the same shape (S "ISR replication", lesson 1).
- **Closed because:** the first generic Raft body sent requests to non-leaders, which this Raft drops; it certified in one run.
- **What it bought us:** `Outcome::Vacuous`, a verdict about the harness. The no-superset-of-a-vacuous-set constraint remains a heuristic (vacuity is not monotone in the fault set).
- **Reopen only if:** never.

Letting a counterfactual replay panic

- **Tried:** Found, not tried: replay feeds recorded inputs to a member whose state has diverged; Raft's leader indexed past its log on an acknowledgement for an entry it never appended in the replayed world, and the panic unwound across the dylib boundary and aborted the process (H §6.4, §9).
- **Closed because:** a replay that cannot run is no evidence, and the abort took the search with it.
- **What it bought us:** `sim::runtime::erase_guarded`; a real-run panic is a test failure; a replay panic is inconclusive and keeps the delivery in the derivation (weaker clause, never a dropped one). Hit 624 times in 365,000 Raft replays.
- **Reopen only if:** never.

Replay shortcuts: earlier-only candidates; unminimized post-`EFF` deliveries

- **Tried:** Round 2: minimize only over deliveries before the output's execution (S "Counterfactual lineage"). Round 5: skip minimizing post-`EFF` deliveries, which are not intervenable, to cut Raft replay cost (H §12 item 4).
- **Closed because:** a later echo can substitute for the original, so the first gave an empty derivation and a false certificate on reliable broadcast; the second made recursion worse, since minimization is also what prunes redundant retransmissions.
- **What it bought us:** candidates include later deliveries; a replayed tick may have nothing to do; a node that cannot reproduce its output with everything kept falls back to recorded lineage (H §6.4). Replay cost is in the recursion, not the per-replay time; caching across shared script prefixes is the open item.
- **Reopen only if:** a variant skips minimization *and* does not recurse through what it keeps.

Maelstrom and partition nemeses for the election-restriction bug

- **Tried:** Round 3 (`e25bfbd2b1`; H §9.1): a Raft-backed `lin-kv` workload, twelve runs, 29 minutes, partitions and latency nemeses, ~1,900 elections, an oracle for an overwritten committed index.

- **Closed because:** the bug needs *selective* loss — `r0` to both followers, `r1` to the old leader, everything else delivered, so logs stay the same length while contents diverge. A partition drops everything on a link; the isolated node falls behind and the index-only check correctly denies it. And `EventualConsistency` + Maelstrom does not type-check: a premise is discharged by a completeness claim, not by "ran 29 minutes, no anomaly" (H §1.1).
- **What it bought us:** the sharp statement of what hydrofrac is for (its output shape, not a race on a bug); a real bug in Hydro's Maelstrom backend; and the honest weak form of the bug-finding claim.
- **Reopen only if:** a Maelstrom nemesis drops individual message identities, which narrows the second point only.

Measurement mistakes: schedule accounting and the lucky baseline

- **Tried:** Round 1's tables charged LDFI 4–8 schedules per hypothesis against one instance per random fault set; the `bern` baseline visited nodes in registration order (S "Measurement grid").
- **Closed because:** hooks under a policy draw no entropy, so one schedule per hypothesis gave every verdict of eight on protocols without schedule-sensitive bugs; and registration order spent the crash budget on the coordinator / replica 0 half the time — the critical node in both protocols. Lucky, not uninformed.
- **What it bought us:** the corrected grid (lineage beats blind everywhere, 2–7× on certificates; beats uninformed random on 2PC at every size; ISR is "not yet a haystack") and the retraction of "random wins instances-to-bug". Raft keeps four schedules because its outcome does depend on the interleaving (H §5.5).
- **Reopen only if:** never — but re-measure before quoting a Raft speed-up.

The undischarged-premise lint

- **Tried:** Round 5 (`09618ccd96`): every search printed every `EventualConsistency` source and whether the specification observed it; the first version printed forty lines for `multi_paxos`.
- **Closed because:** deleted in `aaa0ef79c8`. A report cannot fail a test, and making it one needed "every search over this program" (H §1.2). The direct route was to search every label.

- **What it bought us:** `Spec::all_labels` / `search_all_labels`: one verdict per labelled collection, minted and exit nodes both (they disagree on reliable broadcast), `NotChecked(reason)` for the rest.
- **Reopen only if:** never; `LabelResults` is the lint.

A lattice-aware generated specification for `Singleton` outputs

- **Tried:** Proposed (H §1.2, §12) for ABD's register: observe the final value per correct member, `POST` is equality.
- **Closed because:** `7d58ad20e0`: checking the value re-proves the type system's conclusion. A labelled scalar is a function of its input stream, already vouched for; the only premise left is that every correct member gets the same inputs, so the tap goes on the input stream.
- **What it bought us:** a cleaner class boundary — dissemination, never computation. Scalars are `NotChecked(NotAStream)` until the input tap is built.
- **Reopen only if:** a scalar is labelled over an unlabelled input whose convergence nothing vouches for.

The earlier proposal (`lfdi-on-sim-hooks.md`, `f32054f9`)

- **Tried:** Ordering cuts as "the easy 80%" of faults, a `ChannelHook` on the script-as-schedule model, a `CrashHook` fired `.at_quiescence()`, provenance as "a one-line codegen addition" (S "Appendix").
- **Closed because:** orderings are the fuzzer's dimension and Molly fixed them; crash at a phase barrier cannot express "after prepare to A, before B"; the "one line" is the Level 2 rewrite (26 operators, `wrap_opaque` for `Reference` captures; H §6.3).
- **What it bought us:** composition with the fuzzer — a hypothesis is a fault set, the explorer searches schedules under it, a counterexample is a pair (H §3).
- **Reopen only if:** never.

Unmerged sibling work (`sandbox-6479bd3c…` and relatives, 19 Oct; not on `ldfi-hydrofrac`)

Three further doors, with their own record in `docs/design/ldfi-level-ablation.md` on that branch. The author should confirm which are canonical. - **`Level::Slice` and `Level::Counterfactual` removed** (`4ddea33f19`): `Slice` matched `Execution` and `Counterfactual` matched `Record` on every protocol measured, the latter at 2–10× the replays; `Record` (renamed `Precise`, `fafcd70fe3`, which also undid a wrong

downgrade for programs without provenance) is the default. The same commit made the simulator enforce "every failure ≤ `EFF`" after `4c43956b4b` found it had been firing post-`EFF` crash points the solver never proposed, one of two clause-soundness gaps that certified ISR's bug at three of four levels. [NEEDS AUTHOR REASONING: H §10 says `EFF` "should not shape the design further"; the sibling branch made it the simulator's fault model. Which stands?] - **Disjunctive record provenance** (`73a5db9c05`, flag, default off): all witnesses per record at dedups. Verdicts identical; counts unchanged on 2PC and ISR, *higher* on echo broadcast (10→11, 32→39) — alternatives prune two hypotheses and mint three new crash points. The lazily grown universe already did most of Molly's pruning. Reopen only as "DNF clauses over the single-derivation universe". - **Aggregate-local witnesses** ("any `k` of `n`" at a threshold fold): measurement incomplete at the branch tip. Open, not closed.

Doors we never opened

- **Delay and reordering.** Expressible at the filter as hold-instead-of-drop (H §5.6); reordering is the schedule explorer's dimension.
- **Partitions as a primitive.** Omission sets already express them, and partitions miss selective-loss bugs (H §9.1).
- **Byzantine behaviour, duplication.** Outside Molly's vocabulary; duplication has no simulator hook (H §1.2).
- **`cut_after(n)` on `fail_stop`.** Nothing has needed a reliable channel to fail other than by its sender's crash.
- **The filter under `deploy`.** One OS-chosen interleaving per run, no determinism, replay or barriers (H §11.3): Molly's fixed schedule or worse.
- **Receive-side source for hook-free replay.** Would free lineage from `BatchHook` (H §11.3); deferred, since replay is confined to opaque records.
- **Decision-trace replay.** The general oracle (N §3); worth building only if the strengthened certificates are the ones users want, and it does not repair non-monotone views.
- **Why-not provenance by rewrite.** Molly's route (N §5 (c)); the oracle answers without knowing how the view was computed.
- **Exhaustive schedules for Raft; Raft's `committed` under the generated specification; blind/random on the generic Raft body; a coverage-guided fuzzer baseline; Raft under load.** Each an hour to a day of compute not yet spent (H §9, §10, §12).
- **Membership change.** Nothing to run it on: static membership in the simulator; `dyn_raft` is on `main` (H §12).